

Lösung von AsteroidStatic mit Deep Reinforcement Learning

Task 2 – Deep Reinforcement Learning (Prof. Thomas Nierhoff, OTH Amberg-Weiden)

Autor: Himanshu Mohan Nikam

Matrikelnummer: 25901848

1. Projektüberblick

Das Projekt besteht aus mehreren eigenen Dateien. Die vorgegebene Datei `asteroid.py` wurde nicht verändert:

- `actor_critic.py` – Actor-Critic (One-step, on-policy).
- `DDPG.py` – Deep Deterministic Policy Gradient.
- `TD3.py` – Twin Delayed DDPG.
- `tune_ac.py`, `tune_ddpg.py`, `tune_td3.py` – Optuna-Tuning für die jeweilige Methode.
- die zugehörigen Tuning-Logs, Optuna-Datenbanken (`*.db`), die besten Konfigurationen (`best_*.json`) und die Auswertungsergebnisse (`eval_*_results.json`).

Gemessen werden die beiden geforderten Zielgrößen, jeweils gemittelt über zehn Arenen (Seeds 0 bis 9): erstens die Anzahl der Umgebungsaufrufe bis zum erstmaligen Erreichen des Ziels (Metrik 1, Stichprobeneffizienz) und zweitens die kürzeste erfolgreiche Episodenlänge (Metrik 2, Geschwindigkeit). Zusätzlich wird die Anzahl der Aufrufe festgehalten, bei der diese kürzeste Episode zuerst auftrat. Für das Tuning werden andere Seeds (115 und 119) benutzt als für die finale Auswertung.

2. Ansätze

Die Umgebung besitzt einen kontinuierlichen Aktionsraum, weshalb alle drei umgesetzten Verfahren direkt auf dem Vektor $a = [a_x, a_y]$ arbeiten. Untersucht wurden ein on-policy Verfahren und zwei off-policy Verfahren.

2.1 On-policy: Actor-Critic (in `actor_critic.py`)

Der Actor ist eine Gauß-Politik $\pi(a|s)$, aus der die Aktion gezogen wird; der Critic schätzt den Zustandswert $V(s)$. Beide werden mit dem TD-Fehler $\delta = R + \gamma V(S') - V(S)$ aktualisiert, ergänzt um einen Entropiebonus für die Exploration. Das Verfahren ist on-policy und nutzt keinen Replay Buffer.

2.2 Off-policy: DDPG (in `DDPG.py`)

DDPG verwendet einen deterministischen Actor $\mu(s)$ und einen Aktionswert-Critic $Q(s, a)$. Dadurch ist es off-policy und nutzt einen Replay Buffer sowie Zielnetze mit weichen Polyak-Updates. Die Exploration liefert Ornstein-Uhlenbeck-Rauschen auf der Aktion.

2.3 Off-policy: TD3 (in `TD3.py`)

TD3 erweitert DDPG um drei Punkte: zwei Critics, von denen im Ziel das Minimum genutzt wird (gegen Überschätzung), ein begrenztes Rauschen auf der Zielaktion (Target Policy Smoothing) und seltener aktualisierte Actor- und Zielnetze.

2.4 Weitere erprobte Ansätze

Vor den drei eingereichten Verfahren wurde auch die DQN-Familie umgesetzt und getestet (DQN, Double DQN und Dueling DDQN). Da DQN einen endlichen Aktionsraum benötigt, wurde die Aktion dafür in neun Aktionen diskretisiert, also alle Kombinationen aus $\{-1, 0, +1\}$ für a_x und a_y . Dieser Ansatz löste einzelne Arenen zuverlässig, ist aber nicht Teil der Abgabe, da der Schwerpunkt auf den Verfahren für kontinuierliche Aktionen liegt.

3. Wichtige Techniken

- **Eingangsnormalisierung.** Positionen und Geschwindigkeiten werden anhand der bekannten Grenzen der Umgebung auf einen vergleichbaren Bereich skaliert. Es handelt sich um eine reine Skalierung der Eingänge und nicht um eine Analyse der Umgebung. Ohne diesen Schritt war keines der Verfahren stabil trainierbar.
- **Asymmetrisches Reward-Clipping.** Die Kollisionsstrafe von -100 ist rund zwei Größenordnungen größer als der Fortschrittsterm von $0,5 \cdot \text{Fortschritt}$. Dadurch kollabieren die gelernten Werte auf einen nahezu konstanten negativen Wert, und der Agent kann gute von schlechten Aktionen nicht mehr unterscheiden. Die Strafe wird deshalb nach unten begrenzt, während der Zielbonus von $+10$ unverändert groß bleibt. So bleiben die seltenen erfolgreichen Übergänge das dominierende Signal.
- **Exploration.** Ein rein zufälliger Agent erreicht das Ziel nur in etwa $0,2\%$ der Episoden und auf mehreren Arenen überhaupt nicht. Die Exploration muss daher dauerhaft stark bleiben. Beim Actor-Critic übernimmt das der Entropiebonus, bei DDPG und TD3 das korrelierte Ornstein-Uhlenbeck-Rauschen.
- **Begrenzung der Streuung.** Die Streuung der Gauß-Politik wird über `log_std` gelernt und auf einen festen Bereich begrenzt. Ohne diese Begrenzung wächst sie durch den Entropiebonus unbegrenzt an, bis die Verteilung ungültig wird (siehe Abschnitt 6).
- **Stabilität.** Zusätzlich werden Gradienten begrenzt, Zielnetze verwendet und der Replay Buffer vor dem ersten Update mit zufälligen Übergängen gefüllt.

4. Hyperparameter Tuning

Die Hyperparameter werden mit Optuna gesucht, für jede Methode in einem eigenen Skript und einer eigenen SQLite-Datenbank, sodass unterbrochene Läufe fortgesetzt werden können. Getunt werden unter anderem die Lernraten von Actor und Critic, γ , die Netzgröße sowie die methodenspezifischen Parameter, also τ und die Rauschstärke bei DDPG und TD3 beziehungsweise der Entropiekoeffizient beim Actor-Critic.

Die Tuning-Seeds sind von den Auswertungs-Seeds getrennt. Um geeignete Tuning-Seeds zu finden, wurden die Kandidaten 100 bis 119 zunächst mit einem Zufallsagenten vermessen. Auf den meisten davon wird das Ziel nie zufällig erreicht, sodass das Tuning dort kein verwertbares Signal liefert. Gewählt wurden deshalb die Seeds 115 und 119, auf denen der Zufallsagent das Ziel zumindest gelegentlich erreicht. Die Zielfunktion minimiert die Aufrufe bis zur ersten Lösung; Konfigurationen, die innerhalb des Episodenbudgets nicht lösen, werden ersatzweise nach ihrem geringsten Abstand zum Ziel bewertet, damit die Suche auch dann eine Richtung hat.

5. Ergebnisse

Jedes Verfahren wird mit seiner besten Konfiguration auf den Seeds 0 bis 9 ausgewertet, mit einem Budget von 5000 Episoden je Seed. Tabelle 2 zeigt, wie viele Arenen gelöst wurden und

Tabelle 1: Beste von Optuna gefundene Konfiguration je Verfahren.

Hyperparameter	Actor-Critic	DDPG	TD3
Actor-Lernrate	$1,1 \cdot 10^{-4}$	$8,1 \cdot 10^{-5}$	$1,1 \cdot 10^{-4}$
Critic-Lernrate	$2,6 \cdot 10^{-4}$	$2,3 \cdot 10^{-3}$	$2,2 \cdot 10^{-3}$
γ	0,959	0,957	0,955
τ (Polyak)	–	$1,1 \cdot 10^{-2}$	$1,1 \cdot 10^{-2}$
Entropie β	0,024	–	–
Rauschstärke	–	$\sigma_{OU} = 0,35$	$\sigma_{OU} = 0,17$
Target Policy Smoothing	–	–	0,22 / 0,58
Policy Delay	–	–	3
Batch-Größe	– (online)	128	128
Netzgröße	128	128	128

die Mittelwerte über die jeweils gelösten Seeds.

Tabelle 2: Ergebnisse auf den Auswertungs-Seeds 0 bis 9, gemittelt über die jeweils gelösten Seeds.

Verfahren	Gelöste Seeds	Metrik 1	Metrik 2	Zugehörige Aufrufe
Actor-Critic	7/10	12.157	30,3	45.760
DDPG	8/10	22.654	30,8	47.645
TD3	8/10	21.268	27,6	90.097

Die Mittelwerte beziehen sich jeweils nur auf die von einem Verfahren gelösten Seeds und sind deshalb nicht unmittelbar vergleichbar. Kein Verfahren ist eindeutig am robustesten: DDPG und TD3 lösen je acht der zehn Arenen, der Actor-Critic sieben. Auffällig ist, dass die drei Verfahren auf unterschiedlichen Arenen scheitern (Actor-Critic auf 0, 3 und 6; DDPG auf 3 und 9; TD3 auf 0 und 4); keine Arena bleibt von allen dreien ungelöst. Die Schwierigkeit ist also eher arena- als verfahrensspezifisch. Tabelle 3 zeigt die Einzelergebnisse.

Tabelle 3: Ergebnisse je Seed. Ein Strich bedeutet, dass die Arena im Budget nicht gelöst wurde.

Seed	Actor-Critic		DDPG		TD3	
	Metrik 1	M2	Metrik 1	M2	Metrik 1	M2
0	–	–	43.360	36	–	–
1	235	56	63.899	36	8.582	24
2	3.396	25	9.611	25	4.474	25
3	–	–	–	–	115.471	56
4	67.478	40	38.868	54	–	–
5	6.553	26	3.487	25	15.201	25
6	–	–	14.596	26	14.464	25
7	711	22	4.496	23	2.476	22
8	1.883	20	2.918	21	3.600	21
9	4.842	23	–	–	5.875	23

Betrachtet man nur die fünf Seeds, die alle drei Verfahren lösen (1, 2, 5, 7 und 8), ergibt sich ein klareres Bild. Bei Metrik 2, der Weglänge, liefert TD3 mit im Mittel 23,4 Schritten die kürzesten Wege, gefolgt von DDPG (26,0) und dem Actor-Critic (29,8); die feine Steuerung von TD3 zahlt sich hier aus. Bei Metrik 1, den Aufrufen bis zur ersten Lösung, erreicht der Actor-Critic mit im Mittel 2.556 Aufrufen überraschend den niedrigsten Wert, vor TD3 (6.867) und DDPG (16.882); der hohe Wert von DDPG geht dabei fast vollständig auf die schwierige Arena mit Seed 1 zurück.

Abbildung 1 zeigt beispielhaft die Route eines trainierten TD3-Agenten auf Seed 7.

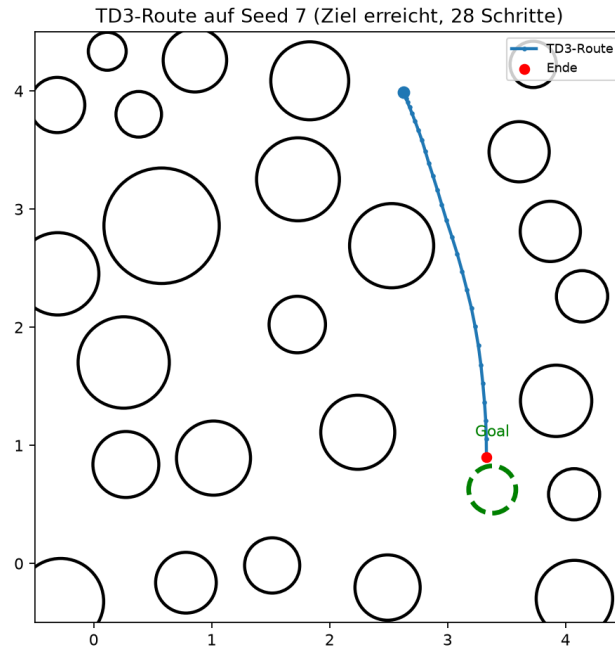


Abbildung 1: Route eines trainierten TD3-Agenten auf Seed 7 (Ziel erreicht nach 28 Schritten).

6. Fehlgeschlagene Versuche

6.1 Symmetrisches Reward-Clipping

Ein erster Versuch begrenzte die Belohnung symmetrisch auf $[-1, 1]$. Damit wurde zwar die Kollisionsstrafe gedämpft, gleichzeitig aber auch der Zielbonus auf denselben Betrag gestaucht. Da das Ziel nur äußerst selten erreicht wird, verlor es dadurch jede Sonderstellung gegenüber einer Kollision, und die Wertschätzungen kollabierten weiterhin. Erst die asymmetrische Variante aus Abschnitt 3 löste das Problem.

6.2 Zu weite Suchbereiche beim Tuning

Ein Tuning-Lauf des Actor-Critic brach nach zwei Trials vollständig ab. Die Suchbereiche für die Lernraten waren zu weit gewählt, sodass Optuna Werte bis 0,245 für den Critic vorschlug. Bei solchen Lernraten divergiert der TD-Fehler, und der Entropiebonus treibt die gelernte Streuung der Politik unbegrenzt nach oben, bis sie numerisch überläuft und die Normalverteilung ungültig wird. Da Optuna Ausnahmen standardmäßig weiterreicht, beendete dieser eine Trial die gesamte Studie. Abhilfe schaffen engere Suchbereiche, die Begrenzung von `log_std` und das Abfangen fehlgeschlagener Trials, damit die Suche weiterläuft.

6.3 Schwache Exploration

Kleines Gauß-Rauschen um eine bereits kollabierte deterministische Politik reicht nicht aus. Der Agent zittert dann nur lokal um den Startpunkt und legt nie genug Strecke zurück, um das Ziel überhaupt zu finden. Notwendig war entweder korreliertes Rauschen oder das Einstreuen von Zufallsaktionen mit voller Magnitude.

7. Diskussion

Die größte Schwierigkeit dieser Aufgabe liegt in der Exploration und nicht in der Lernregel. Das Ziel liegt hinter einem dichten Hindernisfeld, und der Fortschrittsterm der Belohnung zeigt

geradlinig darauf zu. Zustände näher am Ziel tragen dadurch ein höheres Kollisionsrisiko und einen niedrigeren Wert, sodass ein Agent es rational vermeidet, sich dem Ziel zu nähern. Genau das erzeugt die beiden beobachteten lokalen Optima, nämlich die Flucht in den freien Raum und das Verharren am Startpunkt.

Kein Verfahren dominiert in dieser Auswertung. DDPG und TD3 lösen je acht Arenen, der Actor-Critic sieben, und die Ausfälle verteilen sich über verschiedene Arenen. Deutlicher ist der Unterschied bei der Weglänge: TD3 erzeugt die kürzesten Wege, was die feine, abgestufte Steuerung des Verfahrens widerspiegelt. Das Minimum zweier Critics verhindert dabei, dass der Actor überschätzte Aktionswerte ausnutzt, was gerade bei der großen Kollisionsstrafe hilfreich ist. Die getunte TD3-Konfiguration verwendet einen Policy Delay von drei, die verzögerte Aktualisierung des Actors kommt hier also tatsächlich zum Einsatz.

Entgegen der Erwartung, dass ein on-policy Verfahren ohne Replay Buffer bei der Stichprobeneffizienz klar zurückfällt, erreicht der Actor-Critic auf den gemeinsam gelösten Arenen die wenigsten Aufrufe bis zur ersten Lösung. Auf dieser Aufgabe scheinen das starke Reward Shaping und der getunte Entropiebonus den Nachteil der fehlenden Wiederverwendung von Übergängen weitgehend auszugleichen. Erkauft wird dies allerdings mit der geringsten Robustheit und den längsten Wegen: Das Verfahren löst weniger Arenen und findet dort, wo es löst, weniger direkte Wege als die beiden off-policy Verfahren.

8. Fazit

Verglichen wurden ein on-policy Verfahren und zwei off-policy Verfahren für kontinuierliche Aktionen, zusammen mit den Techniken, die das Lernen in dieser Umgebung überhaupt ermöglichen, also Eingangsnormalisierung, asymmetrisches Reward-Clipping und dauerhaft starke Exploration. Alle drei Verfahren lösen sieben bis acht der zehn Arenen; kein Verfahren ist eindeutig überlegen. TD3 findet die kürzesten Wege, der Actor-Critic ist auf den von ihm gelösten Arenen am schnellsten bei der ersten Lösung, aber am wenigsten robust, und DDPG liegt dazwischen. Die verbleibende Schwierigkeit liegt weniger in der Wahl des Verfahrens als in einzelnen, besonders dicht verbauten Arenen.

9. Quellen

Literatur

- [1] Prof. Thomas Nierhoff. *Deep Reinforcement Learning Folie 430*. Vorlesungsfolien. Modul: Reinforcement Learning. 2026.
- [2] Prof. Thomas Nierhoff. *Deep Reinforcement Learning Folie 439*. Vorlesungsfolien. Modul: Reinforcement Learning. 2026.
- [3] Prof. Thomas Nierhoff. *Deep Reinforcement Learning Folie 440*. Vorlesungsfolien. Modul: Reinforcement Learning. 2026.